

TAREA DE CASOS: DISEÑO DEL SISTEMA DE SOFTWARE

Proyecto: SmartRail (Sistema Ferroviario)

Integrantes del Equipo:

Daniela Narváez

Mateo Unda

Anthony Vilaña

José Yáñez

1. Arquitectura del sistema

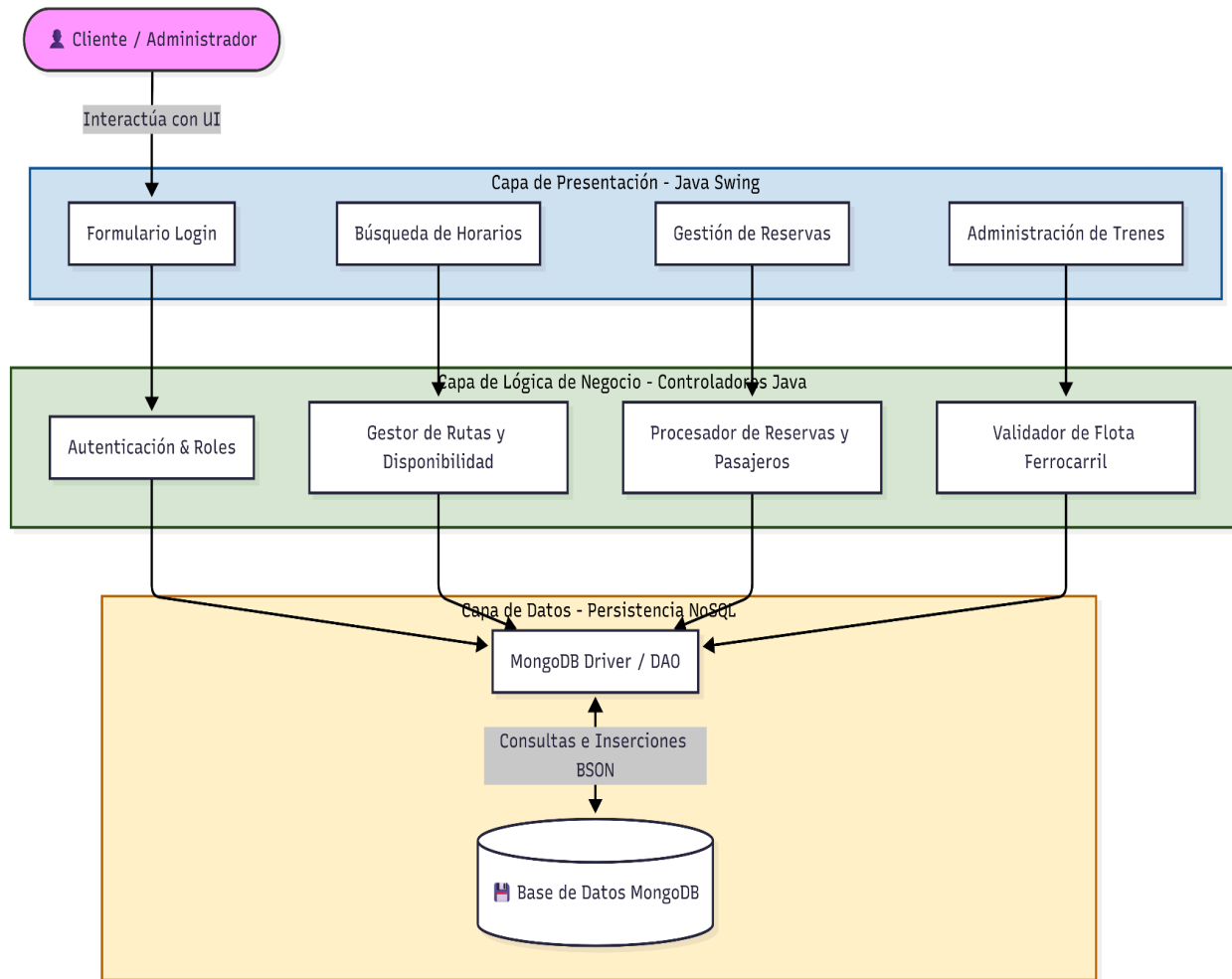
Tipo de Arquitectura Propuesta: Cliente - Servidor

Justificación de la Arquitectura: Para el despliegue del sistema "SmartRail", se ha implementado una arquitectura física de tipo **Cliente-Servidor**. Esta decisión garantiza la centralización, consistencia y seguridad de la información (como evitar la sobreventa de un mismo asiento), permitiendo que múltiples terminales de estación o usuarios operen de forma concurrente sobre una misma fuente de verdad.

El modelo se divide físicamente en dos nodos principales:

- **El Cliente (Aplicación Java Swing):** Representa el entorno local del usuario (ya sea la computadora del Administrador o del Pasajero). Esta aplicación de escritorio asume el rol de "Cliente Pesado", ya que no solo renderiza las pantallas y la interfaz gráfica, sino que también ejecuta las validaciones locales y la lógica de negocio a través de sus controladores, liberando así de carga de procesamiento al servidor.
- **El Servidor (Base de Datos MongoDB):** Representa el nodo central de persistencia. Almacena de forma segura las colecciones de Usuarios, Trenes, Horarios y Reservas.
- **Protocolo de Comunicación:** La aplicación Java (Cliente) se comunica con MongoDB (Servidor) a través de la red mediante el **MongoDB Java Driver**. El cliente envía peticiones de lectura o escritura, y el servidor las procesa y devuelve los resultados en formato de documentos BSON. Esto asegura que, aunque el programa se cierre o la computadora del cliente se apague, toda la información transaccional permanezca intacta en el servidor remoto.

Diagrama de Arquitectura de Capas:



2. Estructura general del sistema

Justificación de uso:

1. Ideal para la Comunicación

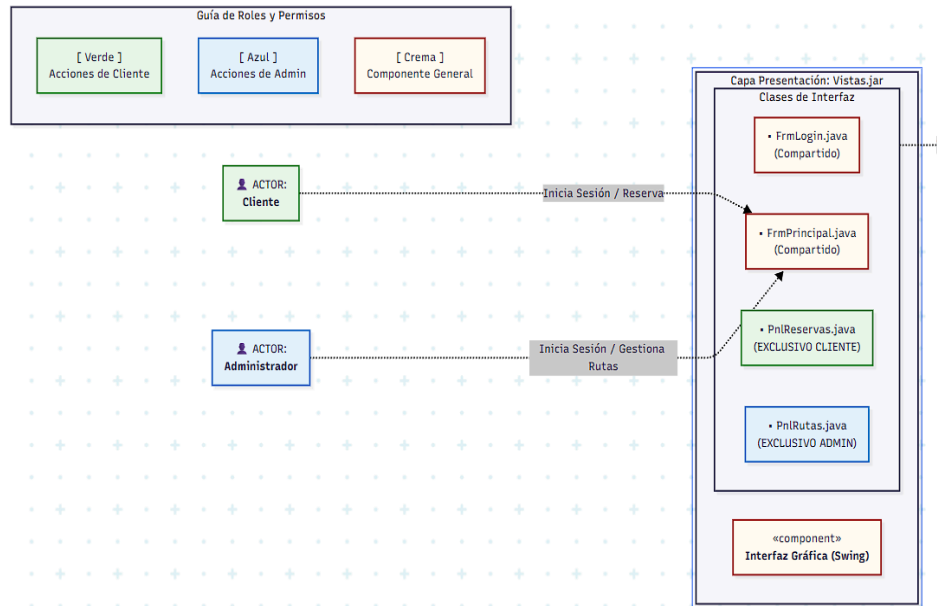
El Diagrama de Componentes eleva el nivel de abstracción (aporta una **vista de pájaro**). Permite explicar el funcionamiento completo del software en menos de 5 minutos, mostrando cómo interactúan los bloques macro del sistema sin perderse en los detalles del código de cada archivo `.java`.

2. Justifica el Control de Roles y la Seguridad por Diseño

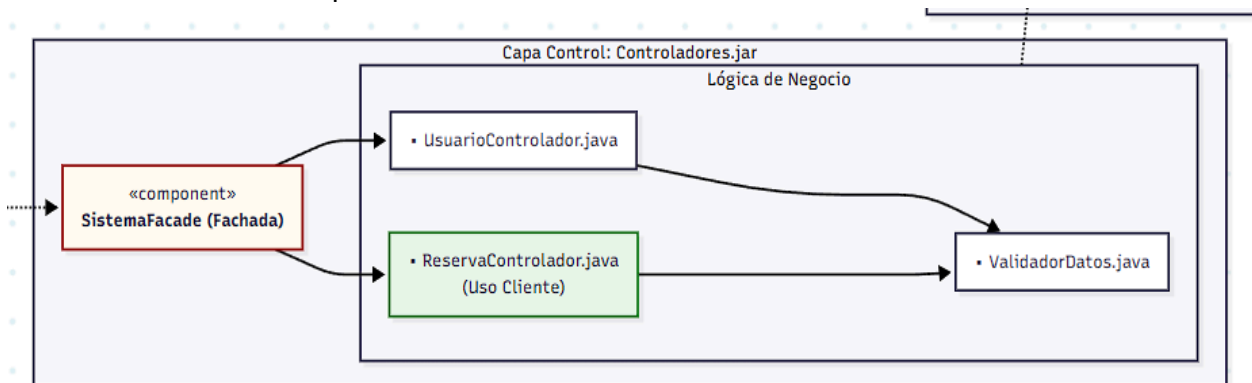
Al modelar componentes y flujos de datos en un solo lugar, el diagrama justifica visualmente por qué un **Cliente** y un **Administrador** pueden usar el mismo programa sin interferir el uno con el otro o los permisos del otro. Permite identificar de un solo vistazo qué módulos son exclusivos (`PnlReservas` vs `PnlRutas`) y cómo el componente central (`SistemaFacade`) actúa como el embudo de seguridad que valida a cada actor antes de dar acceso al componente de persistencia (`Mecanismo DAO`).

Componentes principales

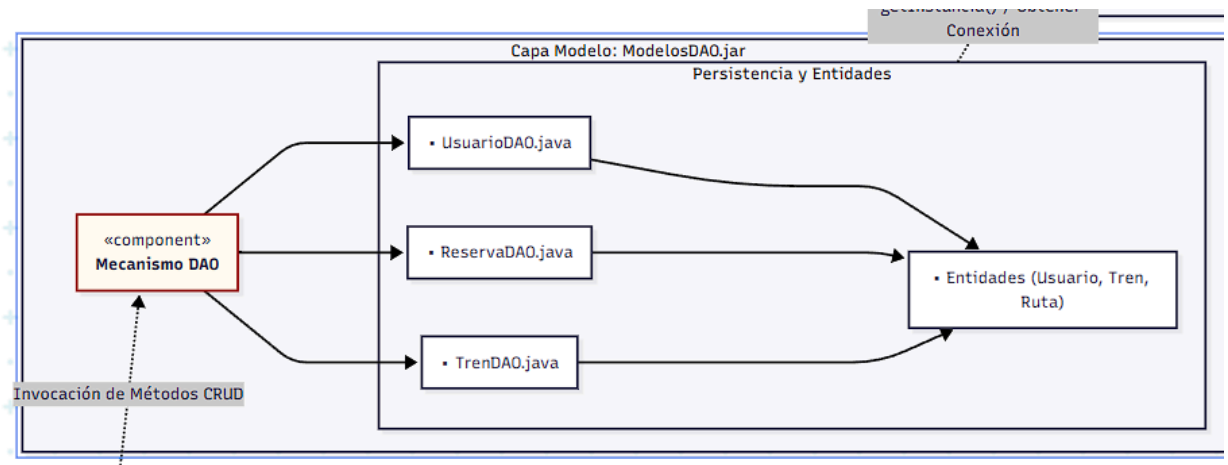
- **Módulo de Interfaz Gráfica (Vistas.jar)**: Es la capa frontal con la que interactúan los actores. Contiene los formularios de acceso (FrmLogin) y la ventana maestra (FrmPrincipal). Para mantener el aislamiento de funciones, las interfaces de usuario están modularizadas en paneles independientes: PnlReservas (exclusivo para Clientes) y PnlRutas (exclusivo para Administradores).



- **Módulo de Control y Fachada (Controladores.jar)**: Actúa como el cerebro del sistema y la capa de reglas de negocio. Centraliza las peticiones mediante el componente SistemaFacade (Patrón Fachada), el cual distribuye el trabajo a controladores especializados como UsuarioControlador (gestión de roles y sesiones) y ReservaControlador (procesamiento de solicitudes). Incluye un componente transversal, ValidadorDatos, encargado de asegurar la integridad de la información antes de ser procesada.



- **Módulo de Acceso a Datos (ModelosDAO.jar)**: Implementa el patrón DAO (Data Access Object). Su única responsabilidad es abstraer y aislar las operaciones CRUD de la base de datos de la lógica de negocio. Utiliza clases del tipo Entidades (Usuario, Tren, Ruta, Reserva) como objetos de transferencia de datos (DTO).



- **Módulo de Conectividad (ConectorBD):** Componente de bajo nivel que gestiona los recursos de red y persistencia. Implementa el patrón **Singleton** (**ConexionBD**) para administrar una única conexión dedicada, apoyándose en el **Driver MongoDB Nativo** para la serialización de datos.

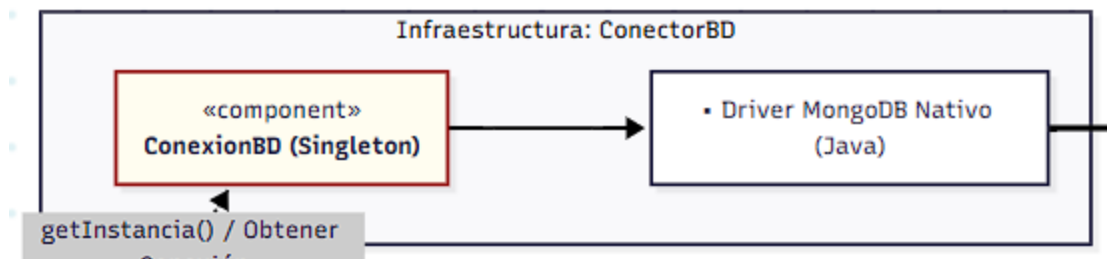
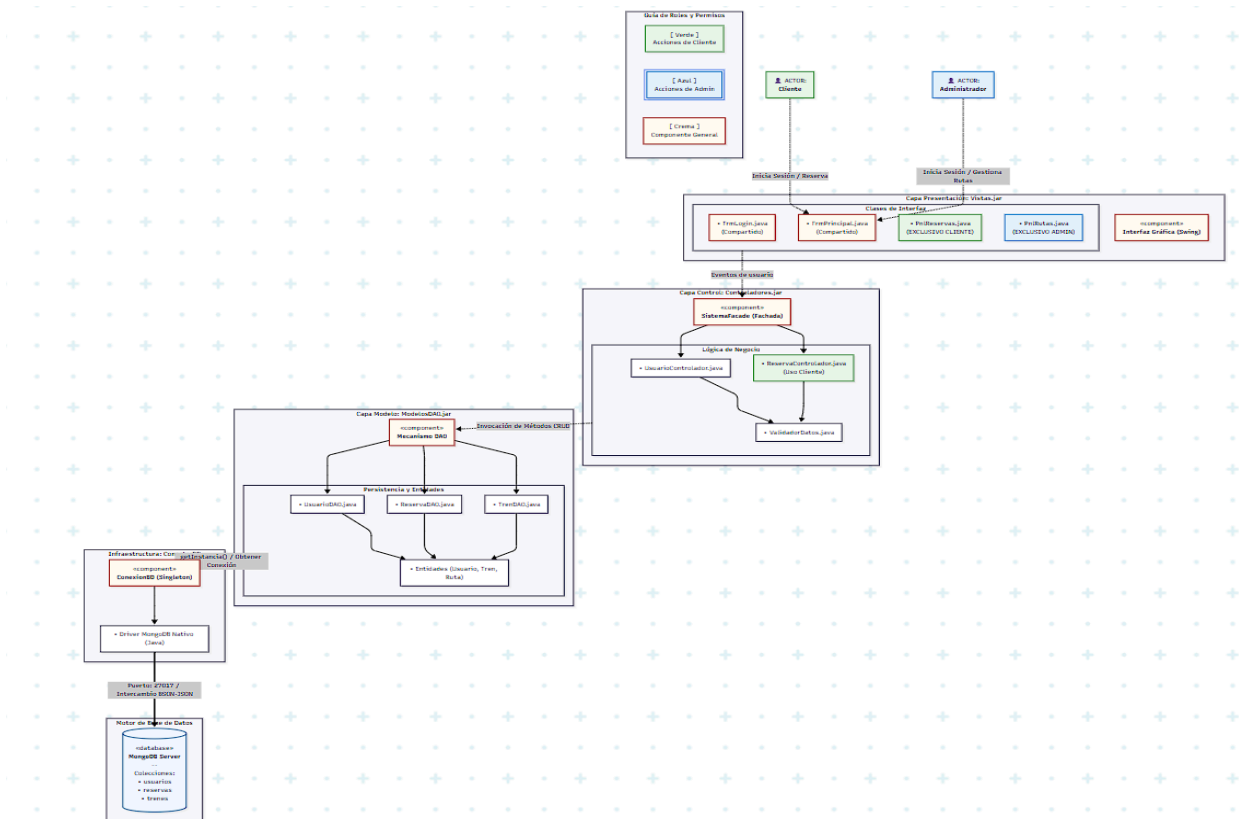


Diagrama de Componentes:



3. Diseño de interfaces

Experiencia de Usuario (UX)

El sistema SmartRail ofrece una experiencia de usuario simple, rápida e intuitiva, orientada a facilitar la gestión de reservas ferroviarias y la administración del sistema.

El usuario puede:

- Iniciar sesión de forma segura
- Navegar mediante un menú principal organizado
- Gestionar información sin dificultad
- Recibir retroalimentación clara en cada acción (confirmación, error, etc.)

La interacción se basa en:

- pantallas claras
- botones bien definidos
- flujos lógicos (seleccionar → confirmar → finalizar)

Ventanas Principales del Sistema

1. Ventana de Inicio de Sesión (Login)

Permite al usuario ingresar al sistema mediante credenciales.

Elementos:

- Usuario
- Contraseña
- Botón “Ingresar”
- Botón “Registrarse” (opcional)

2. Ventana Principal (Menú)

Es el centro de navegación del sistema.

Contiene:

- Acceso a módulos principales
- Barra lateral

Opciones:

- Reservas
- Trenes
- Usuarios
- Reportes
- Cerrar sesión (Salir)

3. Ventana de Gestión de Reservas

Permite realizar y administrar reservas.

Funciones:

- Seleccionar horario o viaje
- Elegir asiento
- Confirmar reserva
- Cancelar reserva

4. Ventana de Gestión de Trenes

Administrada por el personal autorizado (administrador).

Funciones:

- Crear trenes
- Editar información
- Eliminar registros
- Ver estado del tren

5. Ventana de Gestión de Usuarios

Permite administrar los usuarios del sistema.

Funciones:

- Registrar usuarios
- Editar datos
- Eliminar usuarios
- Asignar roles

6. Ventana de Reportes

Presenta información para análisis y control.

Funciones:

- Visualizar reservas realizadas
- Consultar datos del sistema
- Generar reportes

Prototipos / Mockups:

Autenticación del Usuario:

SmartRail

Usuario:

Contraseña:

Ingresar

Registrarse

Usa "admin" en el usuario para acceder como Administrador

SmartRail

Los nuevos usuarios se registran como clientes.
Solo un administrador puede crear cuentas de administrador.

Nombre:

Usuario:

Contraseña:

Guardar

Volver

Vista Cliente:

Menú

Trenes

Reservas

Salir

Trenes y horarios

Origen:

Destino:

Fecha:

Buscar

Limpiar

	Código	Ruta	Fecha	Salida	Llegada	Capacidad	Precio (\$)	Estado
	TR-101	Quito → Guayaquil	2026-06-15	07:00	12:30	45	45.0	Libre
	TR-205	Quito → Cuenca	2026-06-15	10:15	14:00	32	38.5	Libre
	TR-415	Cuenca → Loja	2026-06-17	08:00	15:00	40	60.0	Libre
	TR-520	Riobamba → Manta	2026-06-18	06:30	10:45	25	35.0	Lleno

Reservar seleccionado

June 2026

Anterior

Página 1 de 1 (4 registros)

Siguiente

21

22

23

24

25

26

28

29

30

Menú

Trenes

Reservas

Salir

Mis Reservas

Tren	Ruta	Fecha	Estado	Acciones
TR-101	Quito → Guayaquil	2026-06-15	RESERVADO	Cancelar
—	—	—	CANCELADO	—

Vista Administrador:

Menú

Reservas

Trenes

Reportes

Registrar Tren

Registrar Usuarios

Salir

Reservas (todas)

Usuario	Tren	Ruta	Fecha	Estado	Acciones
Juan Pérez	TR-101	Quito → Guayaquil	2026-06-15	RESERVADO	Reservar Cancelar
Juan Pérez	—	—	—	CANCELADO	Reservar Cancelar
María López	TR-205	Quito → Cuenca	2026-06-15	RESERVADO	Reservar Cancelar

Menú

Reservas

Trenes

Reportes

Registrar Tren

Registrar Usuarios

Salir

Trenes y horarios

Origen: Destino: Fecha: dd/mm/aaaa

BuscarLimpiar

	Código	Ruta	Fecha	Salida	Llegada	Capacidad	Precio (\$)	Estado
	TR-101	Quito → Guayaquil	2026-06-15	07:00	12:30	45	45.0	Libre
	TR-205	Quito → Cuenca	2026-06-15	10:15	14:00	32	38.5	Libre
	TR-415	Cuenca → Loja	2026-06-17	08:00	15:00	40	60.0	Libre
	TR-520	Riobamba → Manta	2026-06-18	06:30	10:45	25	35.0	Lleno

Reservar seleccionado

AnteriorPágina 1 de 1 (4 registros)Siguiente

Menú

Reservas

Trenes

Reportes

Registrar Tren

Registrar Usuarios

Salir

Reportes

Tipo: Generar

Reporte de Reservas

Usuario	Tren	Fecha	Estado	Monto
juan	TR-101	2026-06-15	RESERVADO	\$45.00
juan			CANCELADO	\$
maria	TR-205	2026-06-15	RESERVADO	\$38.50

Menú

Reservas

Trenes

Reportes

Registrar Tren

Registrar Usuarios

Salir

Reportes

Tipo: Generar

Reporte de Trenes

Código	Ruta	Capacidad	Precio	Estado
TR-101	Quito → Guayaquil	45	\$45.00	Libre
TR-205	Quito → Cuenca	32	\$38.50	Libre
TR-415	Cuenca → Loja	40	\$60.00	Libre
TR-520	Riobamba → Manta	25	\$35.00	Lleno

Menú

Reservas

Trenes

Reportes

Registrar Tren

Registrar Usuarios

Salir

Reportes

Tipo: Generar

Reporte de Usuarios

Nombre	Usuario	Rol	Estado
Administrador	admin	Admin	Activo
Juan Pérez	juan	Cliente	Activo
María López	maría	Cliente	Activo

Menú

Reservas

Trenes

Reportes

Registrar Tren

Registrar Usuarios

Salir

Registrar Tren

Código:

TR-XXX

Salida:

HH:MM

Origen:

Llegada:

HH:MM

Destino:

Capacidad:

Fecha:

dd/mm/aaaa

Precio (\$):

Estado:

Guardar

Trenes registrados

Código	Ruta	Fecha	Salida	Llegada	Cap.	Precio	Estado	Acciones	
TR-101	Quito → Guayaquil	2026-06-15	07:00	12:30	45	\$45.0	Libre	Editar	Eliminar
TR-205	Quito → Cuenca	2026-06-15	10:15	14:00	32	\$38.5	Libre	Editar	Eliminar
TR-415	Cuenca → Loja	2026-06-17	08:00	15:00	40	\$60.0	Libre	Editar	Eliminar
TR-520	Riobamba → Manta	2026-06-18	06:30	10:45	25	\$35.0	Lleno	Editar	Eliminar

Anterior Página 1 de 1 (4 registros) Siguiente

Menú

Reservas

Trenes

Reportes

Registrar Tren

Registrar Usuarios

Salir

Editar Tren

Código:

TR-101

Salida:

07:00

Origen:

Llegada:

12:30

Destino:

Capacidad:

45

Fecha:

15/06/2026

Precio (\$):

45

Estado:

Actualizar

Cancelar

Trenes registrados

Código	Ruta	Fecha	Salida	Llegada	Cap.	Precio	Estado	Acciones	
TR-101	Quito → Guayaquil	2026-06-15	07:00	12:30	45	\$45.0	Libre	Editar	Eliminar
TR-205	Quito → Cuenca	2026-06-15	10:15	14:00	32	\$38.5	Libre	Editar	Eliminar
TR-415	Cuenca → Loja	2026-06-17	08:00	15:00	40	\$60.0	Libre	Editar	Eliminar
TR-520	Riobamba → Manta	2026-06-18	06:30	10:45	25	\$35.0	Lleno	Editar	Eliminar

Anterior Página 1 de 1 (4 registros) Siguiente

Menú

Reservas

Trenes

Reportes

Registrar Tren

Registrar Usuarios

Salir

REGISTRO DE USUARIO

Nombre:

Nombre completo

Usuario:

Nombre de usuario

Contraseña:

Contraseña

Rol:

Guardar

Usuarios registrados

Nombre	Usuario	Rol	Estado
Administrador	admin	Admin	Activo
Juan Pérez	juan	Cliente	Activo
María López	maria	Cliente	Activo

4. Diseño de la base de datos

Dado que nuestro proyecto utiliza MongoDB (Base de datos NoSQL orientada a documentos), el modelo clásico de Entidad-Relación no es el más adecuado. En su lugar, presentamos como modelo equivalente un Diagrama Estructural de Clases UML adaptado para NoSQL. Este modelo ilustra nuestras colecciones principales, los tipos de datos BSON y las estrategias de relación mediante referencias (ObjectIds) y documentos embebidos (Arrays).

Para la persistencia de datos se ha implementado una base de datos NoSQL orientada a documentos utilizando MongoDB. Se seleccionó este modelo debido a su alta flexibilidad y escalabilidad. A diferencia de las bases de datos relacionales tradicionales, MongoDB almacena la información en formato BSON (documentos similares a JSON), lo que permite un mapeo mucho más natural y directo con los objetos de nuestro código en Java Swing. Además, nos permite optimizar consultas complejas utilizando la técnica de "documentos embebidos".

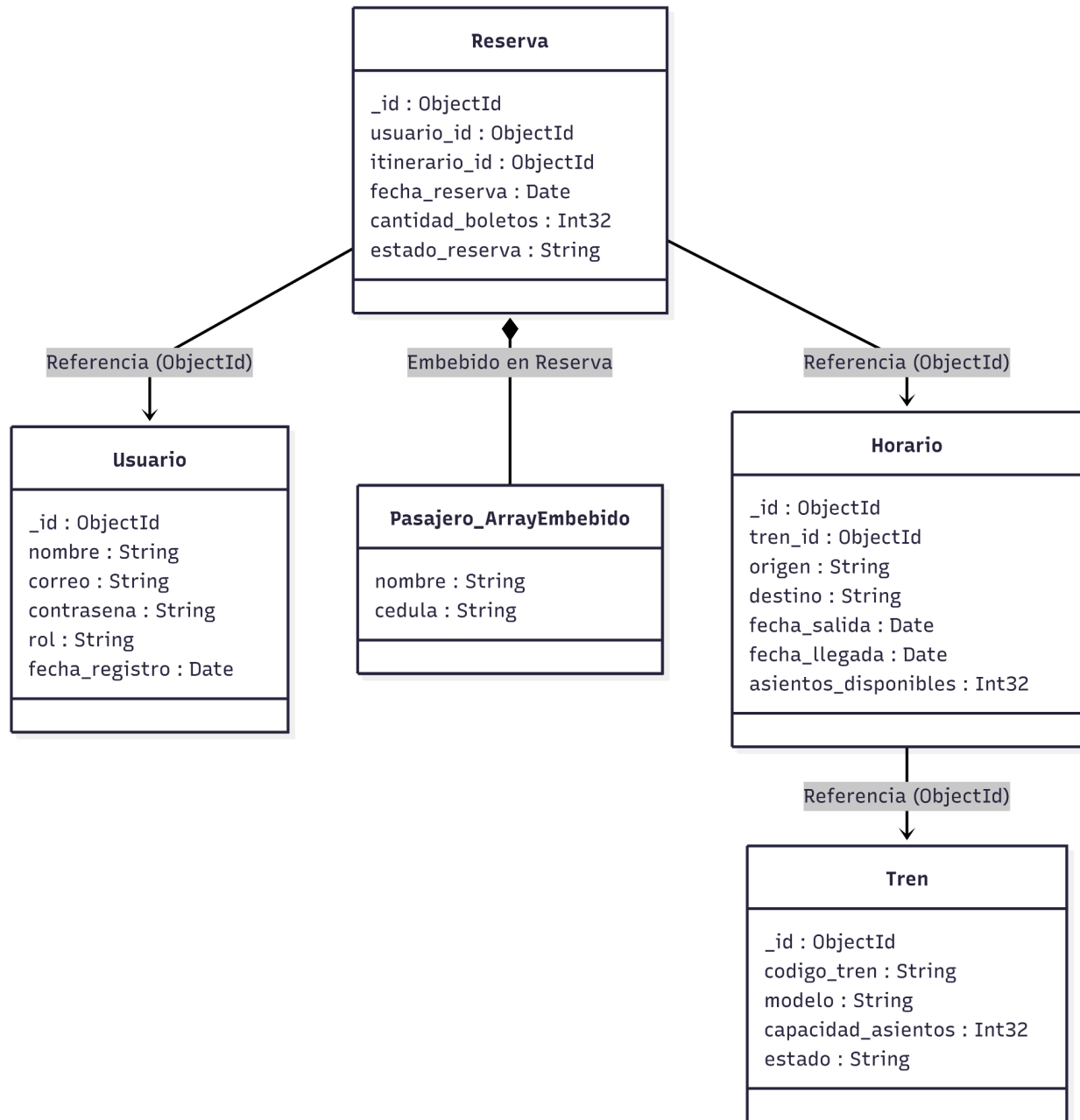
Descripción de Colecciones Principales El sistema manejará la información a través de las siguientes 4 colecciones principales:

- 1. Colección Usuario:** Almacena la información de las credenciales de acceso al sistema. Su propósito principal es gestionar la autenticación y autorización, diferenciando mediante el campo "rol" si quien ingresa es un Cliente estándar o un Administrador del sistema.
- 2. Colección Tren:** Representa el inventario físico de la empresa. Gestiona la flota ferroviaria almacenando datos estáticos como el modelo, el código de unidad y su

capacidad máxima de asientos.

- **3. Colección Horario:** Es la colección que enlaza un tren físico con una ruta y una fecha específica. Es el núcleo del motor de búsqueda para el cliente, ya que lleva el control dinámico de los asientos que van quedando disponibles.
- **4. Colección Reserva:** Es la entidad transaccional más importante. Funciona mediante un modelo híbrido: utiliza referencias (ObjectIds) para enlazarse con el Usuario que compró el boleto y el Horario seleccionado, pero utiliza un **modelo embebido** (un arreglo interno) para guardar los datos específicos de los pasajeros (nombre y cédula) directamente dentro de la reserva, optimizando así la velocidad de lectura de la base de datos.

Diagrama de Clases (Modelo de Datos):



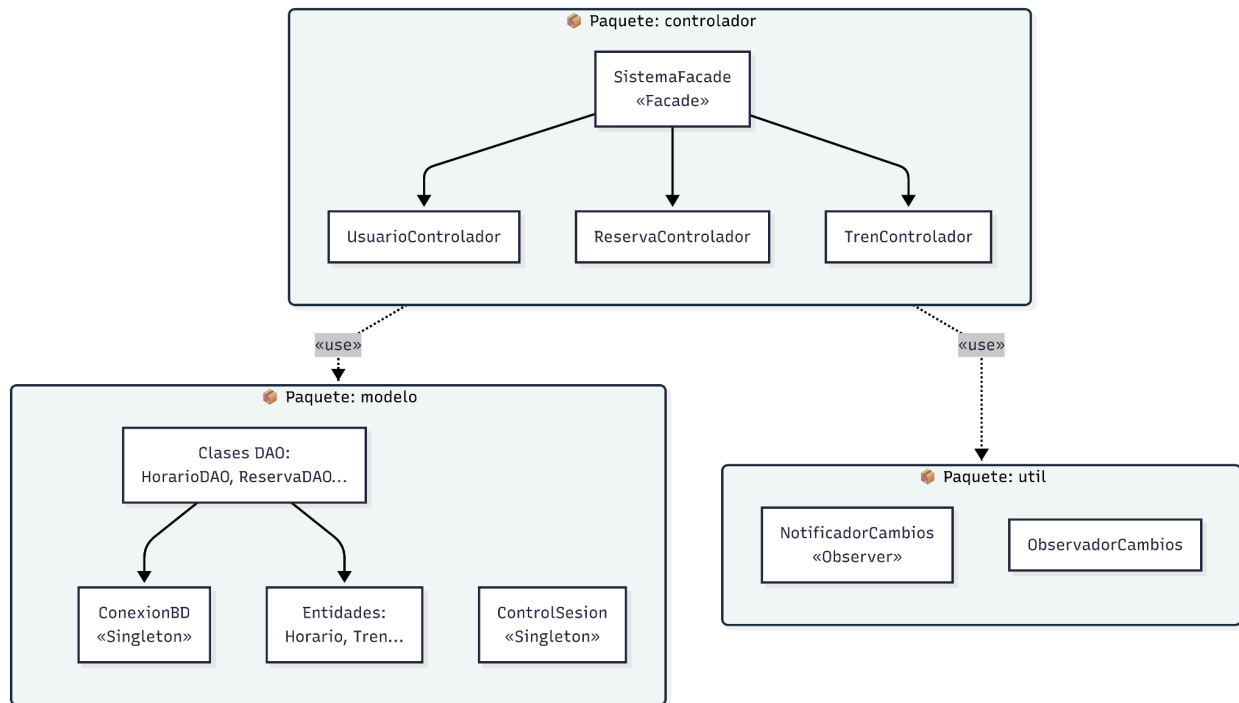
5. Organización de módulos

El sistema de ferrocarriles está estructurado a nivel lógico y de directorios siguiendo un enfoque de arquitectura por capas, separando las responsabilidades de interacción, lógica de negocio y persistencia de datos. Los módulos principales del proyecto se dividen en los siguientes paquetes:

- Paquete controlador:** Centraliza la lógica de negocio y actúa como intermediario. Utiliza el patrón Facade para proporcionar un punto de acceso único y simplificado hacia

los subcontroladores específicos.

- **Paquete modelo:** Contiene las entidades del dominio y la capa de acceso a datos aplicando el patrón DAO (Data Access Object). Además, gestiona de forma centralizada la conexión a la base de datos MongoDB y el estado del usuario mediante un Singleton
- **Paquete útil:** Agrupa herramientas transversales del sistema, implementando el patrón Observer para reaccionar a las actualizaciones de estado en reservas y trenes.



Bibliografía

- Booch, G., Rumbaugh, J., & Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2.^a ed.). Addison-Wesley.